Self-describing GOAL traces for LogGOPSim

DPHPC 2026 progress presentation

Joshua Durrant, Alexander Falter, Samuel Garcin, Mykhaylo Gershman

Supervised by Timo Schneider

17.11.26

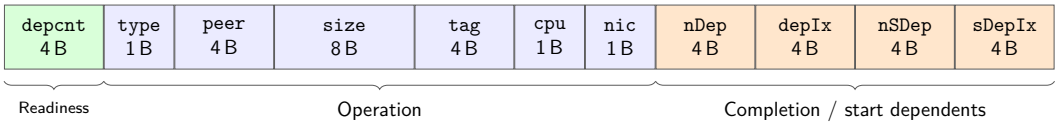
Outline

1. **Task:** the problem, our goals, who did what, structure of our implementation.
2. **Self-describing format:** file layout, encodings, conversion.
3. **Reading and validation:** simulator adapters, errors before the simulation starts.
4. **Case study: ATLAHS:** explicit GPU and message fields, two scheduler fixes.
5. **Fast decoding:** why it matters, one decode plan, five runners.
6. **Benchmarks:** writing (file size and conversion), then reading (all runners, throughput and startup).
7. Next steps and questions.

Task

problem · goals · who did what · structure

The problem: a fixed 39-byte record



- Every node uses every field at full width, whatever values actually occur.
- There is no place for extra information, so producers reuse fields.
- ATLAHS puts the GPU into `nic` and packs five message components into the tag.
- The file doesn't record this: a wrong guess simulates silently with wrong results.

Goals and status

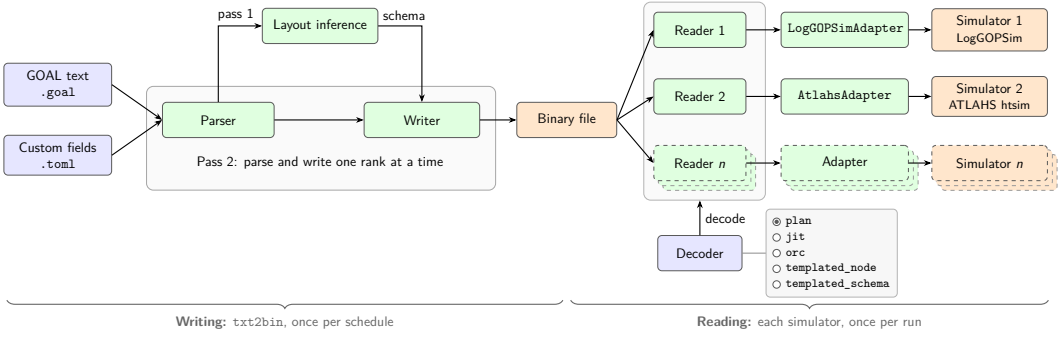
Goal	Question	Status
Self-describing, extensible traces	Can the file describe compact fields and extra attributes?	done
Load-time schema validation	Does the file supply what the simulator needs?	done
Runtime specialization	Can decoding exploit the known schema?	done: five runners
Benchmarking	Smaller and faster than legacy?	preliminary results; full study upcoming

- LogGOPSim and ATLAHS's htsim both read the new format through the same reader.
- Legacy files remain readable; legacy output is still the default.

Who did what

Name	Work
Joshua Durrant	Full benchmarking study (upcoming)
Alexander Falter	OpenMP parallelization and optimizations (upcoming)
Samuel Garcin	Decode replay benchmark, decode plan (optimized interpreter), C JIT with cache, in-process JIT with LLVM ORC, differential test against the generic decoder
Mykhaylo Gershman	Self-describing format and converter, bit packing (including 0-bit constant fields), custom fields, validating reader and simulator adapters, LogGOPSim restructuring, ATLAHS integration, runner architecture and registry, templated runners, test suite (GoogleTest, pytest), benchmark suite and profiler, report

Structure of our implementation

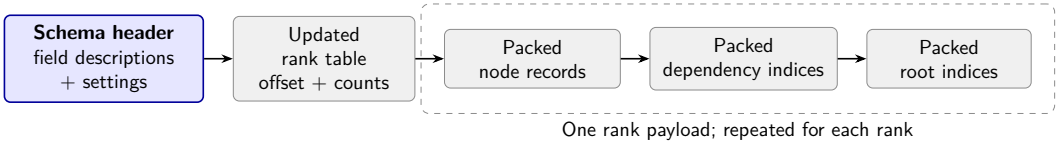


Self-describing format

Mykhaylo Gershman

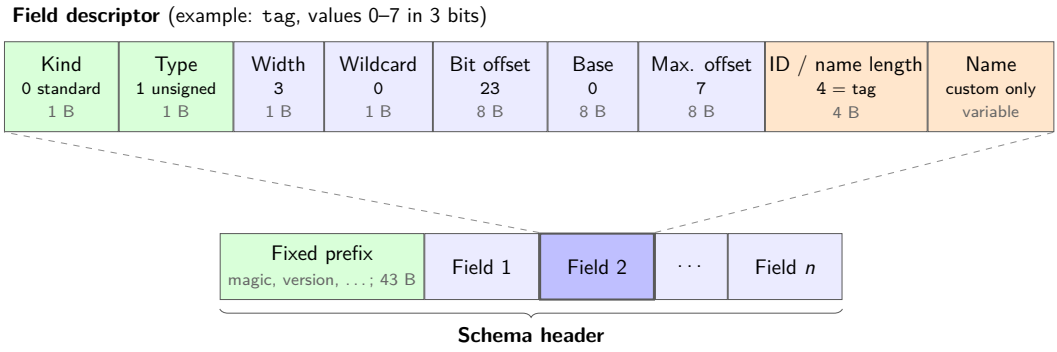
file layout · encodings · conversion

The self-describing file



- Same components as the legacy file (rank table, records, dependency lists, roots), but records and indices use inferred widths.
- New: a **schema header** that describes every field, so a reader needs no built-in knowledge of the layout.
- All records in a file share one layout, so node i of a rank is found directly at `rank_start + i` $\times$ `record_size`, without scanning.
- A new magic number distinguishes it from legacy files, which remain readable.

The header describes every field



- Readers reject unknown versions and type codes, truncated headers and overlapping fields.

Choosing an encoding for the fields

- Each field already gets only as many bits as its largest value needs.
- An **encoding** stores something smaller than the value itself, so it can save even more.

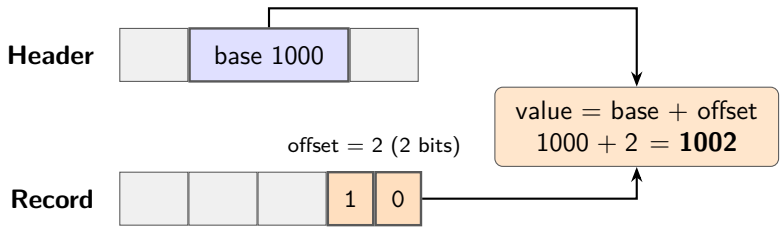
Example 1: one value repeated

1024	1024	1024	1024

Plain value: **11 bits** per value, although every value is the same.

Offset encoding

- The header stores the field's smallest value, the **base**; each record stores only the **offset** from it.



- Example 1: base 1024, every offset is 0, so **0 bits** per value instead of 11.

A case offsets don't fix

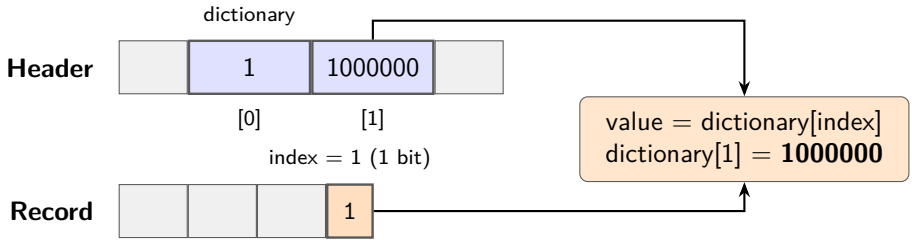
Example 2: two values far apart

1	1000000	1	1000000

- Plain value: **20 bits** per value, to hold 1,000,000.
- Offset encoding: base 1, but offsets still reach 999,999, so still **20 bits**.

Dictionary encoding (proposed)

- The header stores the field's distinct values; each record stores only the **index** into that list.

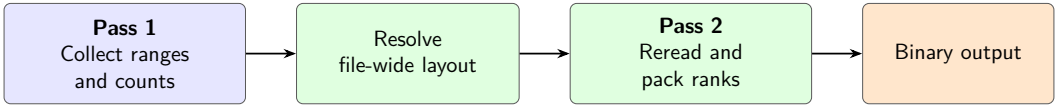


- Example 2: **1 bit** per value instead of 20, at the cost of storing the list in the header.

Combining both

- Offset encoding suits values close together; dictionary encoding suits few values far apart.
- Using both, with the cheaper one chosen for each field, is probably the best approach.
- Offset encoding is implemented; dictionary encoding and the combination are not yet.

Conversion reads the input twice



- The layout must be fixed before the first record is written: if tags 0–7 fit in three bits but a later rank uses tag 1000, earlier records cannot be widened.
- **Pass 1** keeps only file-wide statistics: each field's minimum and maximum, wildcards, dependency counts, the largest rank's node count.
- **Pass 2** parses each rank again and packs its records, dependencies and roots with the final layout.
- Each pass holds one rank's graph at a time, so memory is bounded by the largest rank and no temporary file is needed. The cost is parsing twice.

Making conversion cheaper

Reading and parsing the text twice costs time. Options we considered:

Idea	How	Trade-off	Status
Lighter first pass	Collect ranges and dependency counts without building the graph	Still reads the text twice	In progress: Alexander Falter, on the GPU
Temporary binary file	Parse once, store values in a temporary file, then pack from it	Extra disk space and I/O	Prototype: 1.36× faster on 2.1M nodes, 193 MB temporary data
Keep all graphs	Parse once and keep every rank in memory	Whole schedule must fit in memory	Infeasible for large schedules
Per-rank layouts	Choose a layout for each rank separately	One pass, but needs a format change	Proposed

Switch by input size: keep all graphs in memory when the schedule is small enough, otherwise fall back to a temporary file or two passes. Each option is then used where it is cheapest.

Standard and custom fields

```
# config/custom_fields.toml
[[fields]]
name = "gpu_id"
type = "unsigned"
```

```
l1: send 1024b to 1 tag 42 gpu_id 3
l2: calc 1000 gpu_id 3
```

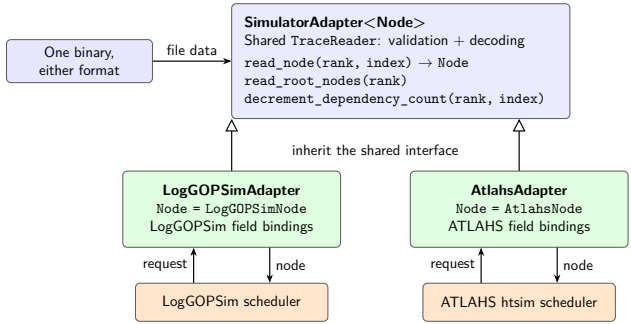
- **Custom fields** are declared in TOML and must appear on every operation.
- From then on they're treated exactly like standard fields.
- **Standard fields** (`type`, `peer`, `size`, `tag`, `cpu`, `nic`) stay standard because they're part of GOAL itself.
- A file can contain fields a simulator doesn't need: each simulator reads only what it binds.

Reading and validation

Mykhaylo Gershman

simulator adapters · load-time checks · backward compatibility

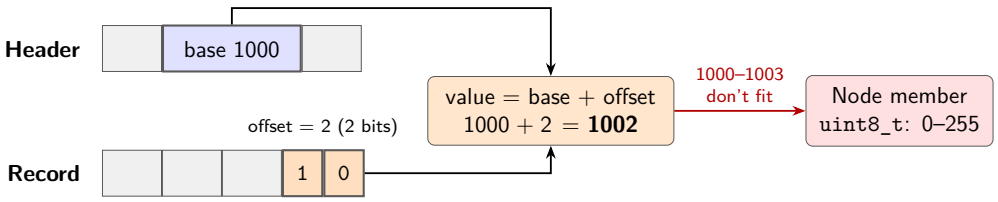
Reading: adapters bind fields to node structs



- Each simulator declares its node type and binds file fields to its members; unbound fields are skipped, so one file serves several simulators.
- The file is memory-mapped; a node is decoded only when the scheduler requests it, and counters are decremented in a private copy, so the file can be reused.

Validation: errors before the simulation starts

At load time, the reader checks that every field the simulator needs exists and that its values fit the simulator's node struct, using only the header.

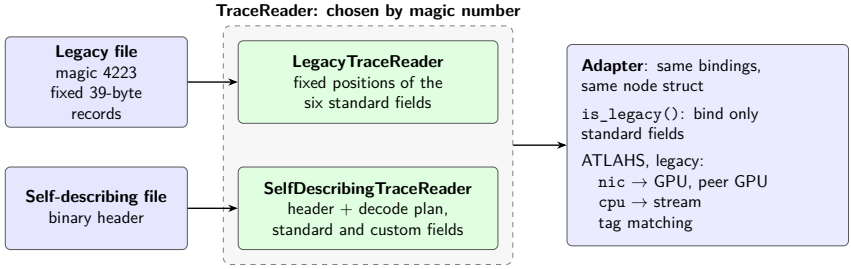


Example: the file says the field holds 1000–1003, but the simulator stores it in a `uint8_t`. Copying 1002 would silently give 234, so the reader refuses the file before reading any node.

When a node is read, cheap checks catch corrupt records (bounds, operation codes, peer ranks).

A missing or incompatible field is a clear error, instead of a silently misread value.

Backward compatibility: one reader, both formats



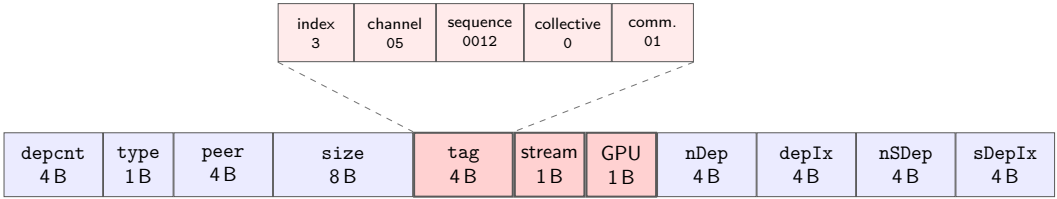
- Existing legacy files run unchanged: the reader detects the format by its magic number.
- The simulators have no legacy code of their own; both formats go through the same field bindings.
- Legacy files have only the standard fields; binding a custom field to one is an error.
- `txt2bin` still writes legacy by default; `--save-mem` still invalidates the file.
- Same output as the original LogGOPSim, with both formats, on every benchmark trace it can run.

Case study: ATLAHS

Mykhaylo Gershman

explicit GPU and message fields · scheduler fixes · results

ATLAHS: GPU and message identity were hidden in other fields



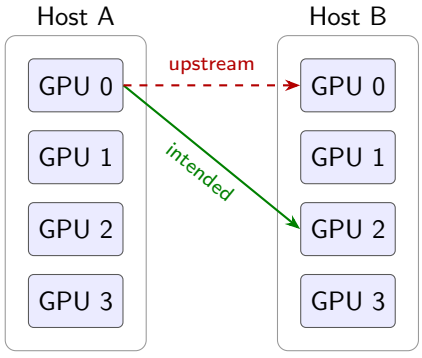
- No GPU field: the GPU goes into `nic`, the stream into `cpu`.
- Five message-identity numbers are glued into the tag as decimal digits.
- The file doesn't say so; htsim guesses, and a wrong guess gives wrong results without an error.

Explicit fields: eight custom fields, no format change

```
before: l2: send 1024b to 1 tag 3050012001 cpu 0 nic 2
after:  l2: send 1024b to 1 tag 0 cpu 0 nic 0 stream_id 0 gpu_id 2 peer_gpu_id 2
            chunk_id 3 channel_id 5 sequence_id 12 message_kind 0 communicator_id 1
```

- **Generator:** new option `--explicit-fields` writes each value as a named field.
- **New: the partner's GPU** (`peer_gpu_id`); the old trace had no place for it.
- **htsim** reads both formats through our `AtlahsAdapter`; its scheduler now uses these fields (next two slides).

Scheduler fix 1: messages went to the wrong GPU



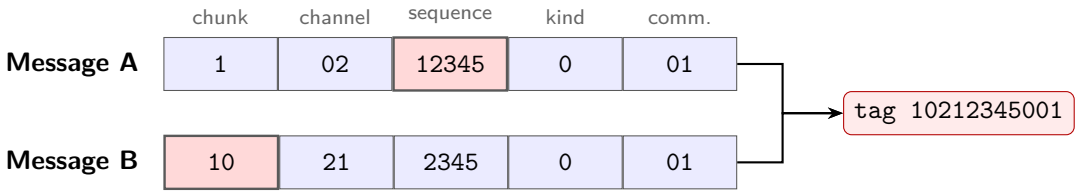
Problem: upstream used the sender's GPU index at both ends. A send from A/GPU 0 to a receive on B/GPU 2 travelled to B/GPU 0, so the wrong network endpoints were simulated.

Fix: we changed htsim to send from the sender's `gpu_id` to the receiver's `peer_gpu_id`. Its queues and completion callbacks now keep both endpoints, so each host updates its own resource clocks.

Also: htsim now tracks execution per `stream_id` (32-bit) instead of the 8-bit `cpu` field; the number of streams comes from the file's schema.

Scheduler fix 2: different messages could get the same tag

Upstream, the generator glued five numbers together as decimal digits to form the tag, and htsim matched sends and receives by that one number:



- **Problem:** a sequence number above 9999 takes five digits instead of four and shifts the rest. Two different messages can then produce the same tag, and htsim may match a receive with the wrong send, without any error.
- **Fix:** htsim now compares the five numbers separately, plus the sender. Explicit-field messages must use tag 0, and a message that never matches stops the run with an error.

ATLAHS: a recorded Llama training run gets 54% smaller

5.7M

operations
4 hosts, 16 GPUs

54%

smaller trace
240.6 → 111.4 MiB

Smaller although the new trace carries all eight ATLAHS fields.

Fast decoding

Samuel Garcin, Mykhaylo Gershman

why · one decode plan, five runners · compiled and templated runners

Why: decoding was up to half of the simulation time

40%

bcast_64k

47%

dissem_16k

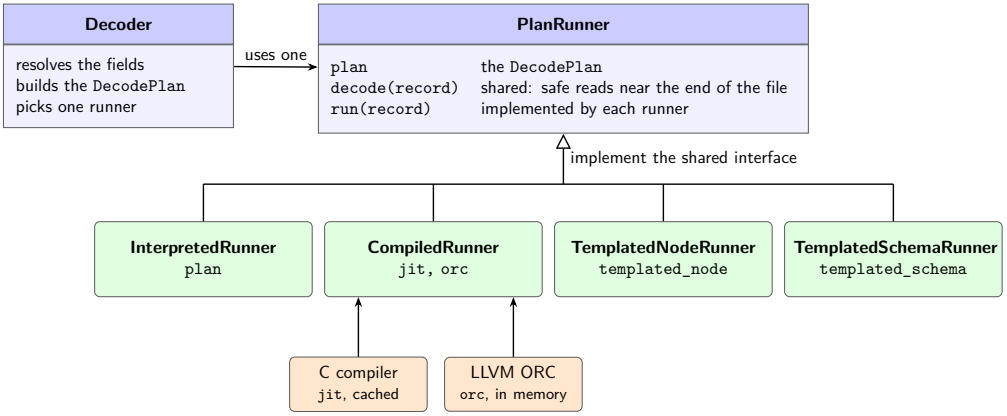
2%

alltoall_256

message matching dominates

Share of the simulation time spent decoding records with the first, generic decoder, which interprets the schema for every field of every node.

One decode plan, five runners



The five runners at a glance

`plan`
Interpreter; default and fallback

Loops over the plan's steps.

+ Works for every trace; needs nothing at setup.
− Reads each step's shift, mask and range at run time.

`jit`
Generated C, compiled with `cc`

Generates C for the schema, compiles it into a library and caches it.

+ Every position and mask is a constant.
− Needs a C compiler; compiles on first use.

`orc`
In-process JIT with LLVM

Builds the same function in memory with LLVM ORC.

+ No compiler process, no files.
− Needs LLVM; compiles on every run.

`templated_node`
Template per simulator node type

Fixes the field count and store widths at compile time.

+ No compiler at run time; works for every trace.
− Positions and masks still come from the plan.

`templated_schema`
Lower bound for comparison

The generated C's code, compiled into the simulator ahead of time.

+ The best a specialized runner can do: every constant known, nothing compiled at run time.
− Only for layouts compiled in, so not a choice for new traces.

Compiled runners hard-code the schema

```
int decode(const u8* r, u8* const* t) {
  u64 w, c;
  // type: 2 bits at bit 1, base 1
  memcpy(&w, r + 0x0, 8);
  c = (w >> 0x1) & 0x3;
  if (c > 0x2) return 0;
  c += 0x1;
  store8(t[0] + 0x38, c);
  // peer: 16 bits at bit 3
  memcpy(&w, r + 0x0, 8);
  c = (w >> 0x3) & 0xffff;
  store32(t[0] + 0x3c, c);
  // size: 0 bits, constant 1024
  store64(t[0] + 0x40, 0x400);
  // ... 8 further fields
  return 1;
}
```

Generated C for bcast_64k, abridged.

- Every position, mask and range is a constant: no loop, no dispatch; a 0-bit field is a single store.
- **`jit`:** compiled with `cc`, cached on disk.
- **`orc`:** the same function compiled in memory with LLVM, loaded as a plugin only when used.
- Only numbers from the header go into the code, so a trace cannot inject code.

Preliminary benchmarks

Samuel Garcin, Mykhaylo Gershman

writing: size and conversion · reading: throughput and startup

One laptop (Apple M3 Pro), few repetitions. Joshua Durrant will run the full benchmarking study.

Writing

GOAL text $\rightarrow$ binary trace

file size · conversion time

Writing: file size is 26–93% smaller

Trace	Nodes	Record	Legacy → new	Smaller
bcast_8	14	39 → 2 B	808 → 594 B	26%
bcast_1k	2k	39 → 4 B	110 → 30 KiB	73%
bcast_64k	131k	39 → 4 B	6.9 → 1.9 MiB	73%
wide_64k (random 64-bit values)	131k	39 → 15 B	6.9 → 3.2 MiB	53%
alltoall_256	131k	39 → 2 B	5.4 MiB → 404 KiB	93%
ring_256	261k	39 → 4 B	10.7 → 1.3 MiB	88%
dissem_16k	459k	39 → 4 B	19.2 → 2.4 MiB	88%
ATLAHS Llama run (8 extra fields)	5.7M	39 → 17 B	240.6 → 111.4 MiB	54%

- Narrow or constant fields shrink most; even random 64-bit values save half.

Writing: conversion time

Complete `txt2bin` run on the same GOAL text, **ms**.

Trace	Original `txt2bin`	Ours, legacy	Ours, new format
bcast_8	53	51	**3.3**
bcast_1k	63	119	**7.3**
bcast_64k	758	4527	**247**
dissem_16k	362	1462	**333**
ring_256	**138**	187	252
alltoall_256	**74**	136	82
wide_64k	779	4571	**295**
atlahs_64k (8 extra fields)	n/a	n/a	585
ATLAHS Llama run	6 500	n/a	19 500

- New format: faster than the original on most traces despite two passes.
- Our legacy writer remaps the file per rank: up to 6× slower; to be fixed.

Median of 5 interleaved runs. Legacy converters end with a system-wide `sync()` (≈50 ms). n/a: legacy cannot hold the custom fields (ours also rejects the Llama run's >32-bit tags). ATLAHS: one run.

Reading

binary trace $\rightarrow$ simulation

decoding · access pattern · startup · complete simulations

Reading: decoding throughput

Read every node once, **ns per node**; setup and compilation excluded.

Trace	Nodes	plan	jit	orc	templated_ node	lower bound (templ._schema)
bcast_8	14	30.2	**17.8**	**17.9**	24.1	17.4
bcast_1k	2k	27.5	**13.9**	**13.8**	21.2	13.3
bcast_64k	131k	26.5	**13.4**	**13.4**	21.1	13.2
dissem_16k	459k	29.2	**12.9**	**12.8**	24.0	12.4
ring_256	261k	30.2	**12.7**	13.3	24.0	13.1
alltoall_256	131k	29.0	**12.4**	**12.4**	24.1	11.9
wide_64k	131k	26.4	**13.7**	**13.8**	21.1	13.5
atlahs_64k	131k	41.2	17.2	**16.6**	29.9	16.3

`jit` and `orc` come within 6% of the lower bound: generated code is no longer the limit.

Median of three repetitions. Bold: fastest runner (within 3%).

Reading: the simulator's access pattern

Replay of LogGOPSim's reads and counter updates, without the network model; **ns per node**.

Trace	Legacy	plan	jit	orc	templated_ node	lower bound (templ._schema)
bcast_8	131	**148**	159	172	157	151
bcast_1k	27.3	39.7	**34.0**	**33.8**	37.5	33.6
bcast_64k	26.3	39.0	**32.9**	**32.6**	36.8	32.6
dissem_16k	26.4	35.6	**30.6**	**30.5**	33.4	30.4
ring_256	22.6	36.2	**31.7**	**31.7**	34.3	31.3
alltoall_256	7.4	**13.5**	**13.4**	**13.5**	**13.5**	13.5
wide_64k	26.1	39.8	**33.7**	35.8	41.1	35.7

`jit` and `orc` reach the lower bound; the remaining 4–9 ns gap to legacy lies in record and dependency-list access, not in decoding.

Median of three repetitions; bcast_8 is dominated by fixed costs. Bold: fastest runner for the new format (within 3%).

Reading: startup cost

Setup with an **empty JIT cache**, then read every node once; no network simulation; **ms**.

Trace	plan	jit	orc	templated_ node	lower bound (templ._schema)
bcast_8	**0.006**	329	2.6	**0.006**	0.006
bcast_1k	0.074	334	3.0	**0.066**	0.048
bcast_64k	4.2	350	5.4	**3.5**	2.4
dissem_16k	13.9	345	**8.8**	11.6	6.1
ring_256	8.0	332	7.3	**7.0**	3.8
alltoall_256	3.8	347	4.4	**3.2**	1.6
wide_64k	4.2	342	5.7	**3.4**	2.4
atlahs_64k	6.1	338	6.1	**4.6**	2.8

Compiling costs 330–350 ms for `jit` and about 2.5 ms for `orc`; `templated_node` needs none and wins most cold starts.

Median of three repetitions. Bold: fastest runner (within 3%).

Reading: complete simulations

Complete LogGOPSim runs including startup, **ms**; lower is better.

Trace	Original LogGOPSim	Ours, legacy	plan	jit (cached)	orc	templated_ node	lower bound (templ._schema)
bcast_8	3.1	3.3	3.3	3.6	12.8	3.4	3.2
bcast_1k	3.8	3.9	3.9	4.1	13.2	3.9	3.8
bcast_64k	48.4	43.9	45.1	43.7	52.9	44.1	43.6
dissem_16k	146.4	122.7	116.6	110.8	118.3	114.3	110.2
ring_256	39.8	32.8	34.5	32.9	41.8	33.6	32.4
alltoall_256	1950	2004	1997	1996	2010	1999	1998
wide_64k	crashes	78.4	79.6	79.5	88.3	81.3	77.8

- Larger collectives: all runners but `orc` (loads LLVM in every run) are 7–24% faster than the original.
- alltoall_256: message matching dominates, at most 3% slower. Results identical everywhere; the original crashes on wide_64k.

Median of 21 runs, configurations interleaved in a seeded random order. Original: last upstream commit (76451de), same build type, on the legacy files.

Which runner wins where

Tiny trace	Cold start, small/medium traces	Cold start, `dissem_16k`	Long read phase
`plan`	**`templated_node`**	**`orc`**	**`jit` ≈ `orc`**
nothing to amortize	no compilation	compilation pays off	similar throughput

- Lower bound (`templated_schema`): after setup, `jit` and `orc` come within 6% of it; at startup, the best runner needs 1.4–2× its time.
- Measured on one laptop; automatic selection needs crossover points on more layouts.

Next steps

Reading

- Select the runner using startup cost, trace size and cache availability.
- Profile the remaining rank-lookup, packed-index and counter-update costs.

Writing

- Make legacy I/O comparable: per-rank remapping and flush policy.
- Measure the phases, then evaluate a lean analysis pass or dictionary encoding.

Evaluation

- Euler study (Joshua): Linux/x86-64, cold file caches and peak memory.
- Larger recorded workloads and custom-field-heavy traces.
- Parallelization experiments (Alexander), keeping cross-format and cross-runner checks.

Questions

- **Runner selection:** should cold-start cost or repeated-read throughput guide the default? Keep an explicit override?
- **Writing:** after matching I/O policies, should we prioritize a cheaper analysis pass or dictionary compression?
- **Evaluation:** which larger workloads and cold-cache scenarios should the Euler study prioritize?